

Graph Card Control: Evaluating Search Agents in a Stochastic Abstract Strategy Game

Omri Avital (208693341) Itai Reder (318850781)

237-2-5513 Search Methods in Artificial Intelligence

Abstract

We introduce Graph Card Control, a configurable two-player, stochastic, perfect-information game for controlled comparison of bounded search implementations and transparent baselines. The only random event is replacement of a selected card in a three-card public market. We evaluate random, greedy, rule-based, Monte Carlo tree search (MCTS), and depth-2/depth-3 expectimax agents across three board geometries, three stopping rules, player order, and independently varied deck composition. The corrected suite contains 1,116 replay-validated games and prevents agents from conditioning on hidden deck order. Rule achieved the highest aggregate win-points rate (66.0%) at 4.8 ms per decision. Among search agents, Expectimax d3 reached 63.6% at 956.3 ms. Depth-3 expectimax cost 8.6 times depth 2 and obtained 52.8% in their direct matches. Board geometry, stopping rule, deck composition, and player order all changed outcomes. The results characterize these fixed-budget implementations; they are not universal algorithm rankings.

Introduction and Literature Review

Game playing provides a controlled setting in which states, actions, stochastic transitions, and utility can be specified exactly. This project asks a narrower empirical question: when an unfamiliar game combines adversarial choice with a public stochastic card refill, how do bounded search agents compare with transparent baselines under changes to geometry, stopping rule, deck composition, and move order? The course explicitly identifies a custom strategy game with search-based and threshold agents as a suitable research direction.

Graph Card Control is a stochastic, perfect-information game. Exact lookahead therefore requires decision nodes for both players and expectation over replacement-card outcomes. Ballard’s *-minimax analysis showed how bounds can prune trees containing chance nodes (Ballard 1983), while Monte Carlo *-minimax later combined sparse sampling with stochastic adversarial search (Lanctot et al. 2013). These works motivate expectimax as the structurally appropriate exact model and also predict its computational difficulty when both action and chance branching are present.

MCTS offers an alternative allocation of computation. Coulom (2006) integrated Monte Carlo evaluation with selective tree growth, and Kocsis and Szepesvari (2006) introduced UCT, establishing consistency and finite-sample error

bounds for Monte Carlo planning. Sparse sampling can avoid enumerating every stochastic successor (Kearns, Mansour, and Ng 1999). Our corrected MCTS samples replacement cards from the public composition on every traversal, aggregates outcomes on stochastic action edges, and treats the opponent as a minimizing player. Expectimax instead enumerates the complete known refill distribution after selecting its bounded action beam.

The broader MCTS literature separates selection, expansion, simulation, and backup, with playing strength depending on choices in every phase (Browne et al. 2012). This matters for interpretation: our comparison tests one explicit budget and rollout design, while expectimax spends computation on exhaustive chance backup inside a depth limit. A result for either implementation is therefore evidence about a cost-allocation choice, not about all members of the algorithm family.

Deeper search is not automatically better when leaf evaluation is approximate. Pearl (1983) demonstrated that repeated minimax backup can amplify evaluation noise and, in pathological cases, degrade decisions with depth. The choice-function framework likewise formalizes when pruning the considered action set can preserve policy improvement (Issakimuthu, Fern, and Tadepalli 2020). These results are directly relevant to our top-2 expectimax cap and to interpreting the observed depth/runtime trade-off rather than treating depth as an unconditional quality scale.

We organize the evaluation around five questions. Which agent is strongest across the chosen boards and stopping rules? How much decision time does that performance require? Does moving first change the comparison? Does independently changing the deck alter the ranking? Finally, do deeper expectimax and the net-launcher mechanic produce evidence of qualitatively different play? These are descriptive questions about the implemented system; they are not claims of universal algorithm superiority.

General game playing emphasizes evaluating agents on games not engineered around a single solver (Genesereth, Love, and Pell 2005). Our contribution is accordingly methodological rather than algorithmic: (i) an executable stochastic game with configurable geometry, objectives, decks, and stopping rules; (ii) a reproducible matched-order experiment system; and (iii) an empirical comparison that includes computational cost and behavior, not only win rate.

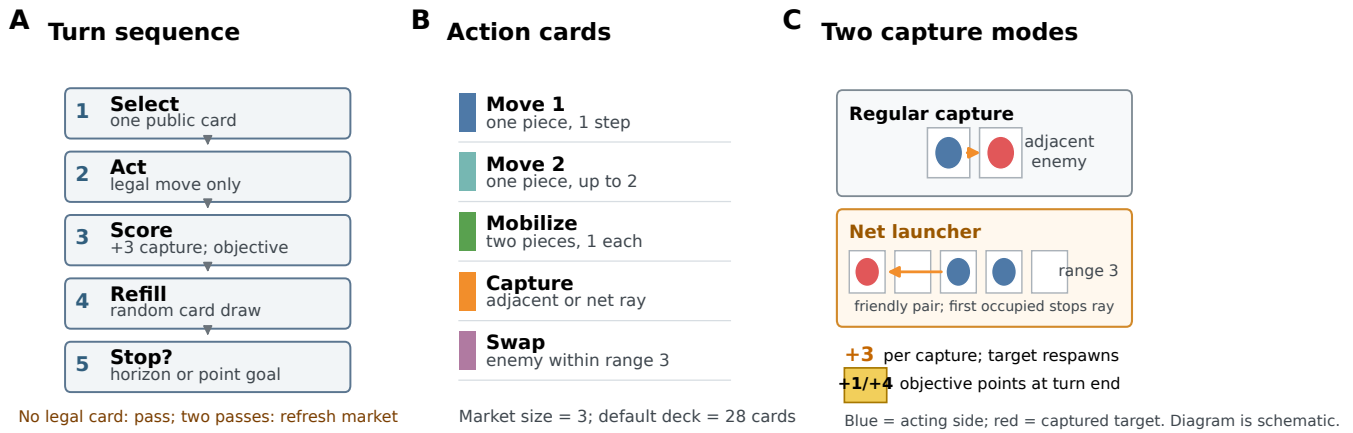


Figure 1: **Turn structure and capture modes.** (A) The player selects a public card, acts, scores, refills the market randomly, and tests the stopping rule. With no legal card the player passes; two passes refresh the market. (B) The five card types and their ranges. Mobilize moves two pieces when possible and otherwise one. (C) A regular (melee) capture targets an orthogonally adjacent enemy. A net launcher uses an adjacent friendly pair and the first occupied cell on an outward range-3 ray. Both score three points and respawn the target; objective cells score at turn end. Blue is the acting side and red the target; diagrams are schematic.

Methodology

Game model

Players alternate turns on an orthogonal grid and control three pieces each. The current player selects one legal action from a public three-card market. Movement cards reposition pieces; Capture scores three points through an adjacent attack or net launcher; Swap exchanges a friendly and enemy piece within Manhattan distance three. Captured pieces respawn, preventing elimination. Objective cells score at the end of every occupying turn. After a played card is discarded, its market slot is refilled by a draw from the known remaining composition. Thus the state is fully observable while the next refill is stochastic (Figure 1).

A pass occurs only when all visible cards are unusable. Preliminary experiments identified repeated passes as an absorbing market failure, so every final condition refreshes all three market cards after two consecutive passes. Any legal move resets the counter. This rule change is fixed across the final suite and is not itself an experimental factor.

Implementation

The Python 3.10+ rules engine represents a state as an immutable record and applies actions through a shared transition function. Random-number-generator state is carried inside the game state, making a game deterministic once its seed is fixed. Legal-action generation and state transitions are shared by human, baseline, and search agents, preventing agent-specific rule implementations. Runtime dependencies and analysis libraries are declared in the repository requirements file.

Replacement draws can be forced during search and replay. Agents receive the executable state, which retains a shuffled deck for deterministic match execution, but corrected successor evaluation never consults that hidden ordering. Greedy

and rule-based evaluation use a deterministic most-probable public refill solely for action ordering. Expectimax backs up the exact probability-weighted value of every remaining card type; its top-2 beam is ordered in the appropriate maximizing or minimizing direction. MCTS resamples the public draw distribution with an agent-local random generator on every action-edge traversal and alternates root-player maximization with opponent minimization (100 simulations, rollout depth 10, exploration constant 1.4). Reversing the hidden deck while holding all public fields fixed changed zero tested choices for greedy, rule, expectimax depth 2, and MCTS.

Boards, decks, and stopping rules

The main factorial varies three symmetric boards (Figure 2) and three stopping rules: a fixed 25 turns per player, first to 10 points, and first to 40 points. Goal games retain a 50-turn-per-player safety cap. Goal 10 emphasizes openings, whereas goal 40 permits sustained positional control. The inner-ring center awards four points, deliberately making center occupation more valuable than a three-point capture.

Deck composition is tested independently on the default board at the fixed horizon. The 28-card control deck contains 8 Move 1, 6 Move 2, 5 Mobilize, 5 Capture, and 4 Swap cards. Movement-, capture-, and swap-heavy variants alter these counts without changing card behavior. This reduced design avoids an uninterpretable full cross-product while preserving a common control.

Agents

Random samples uniformly from legal actions. Greedy selects the immediate public-information successor with the highest shared heuristic. Rule prioritizes captures and objectives, then uses that heuristic for tie-breaking. Expectimax alternates maximizing and minimizing decision nodes and expands exact replacement-card probabilities at depth 2 or

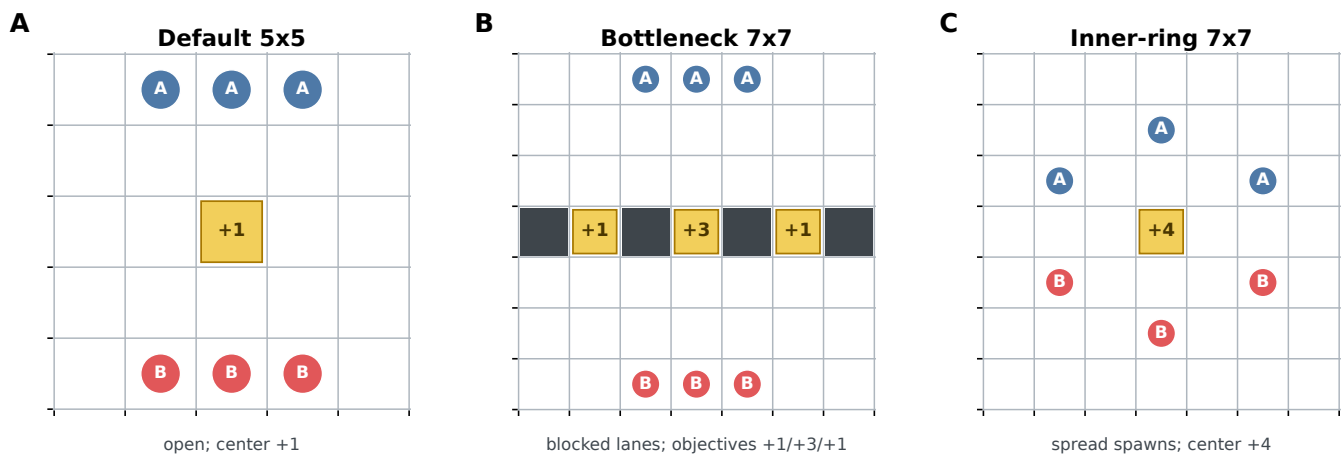


Figure 2: **Three board geometries define the spatial experimental factor.** (A) Default open 5x5 board. (B) Bottleneck 7x7 board with blocked central lanes. (C) Inner-ring 7x7 board with more widely separated spawns and a four-point center. Blue and red circles mark the initial cells of players A and B; yellow cells are end-of-turn objectives labeled by points; dark cells are blocked. Coordinates and piece identities are omitted because the JSON board definitions are the authoritative executable specification.

3; its top-2 beam keeps the best root-player actions at maximizing nodes and the strongest replies at minimizing nodes. MCTS uses 100 simulations, rollout depth 10, sampled public chance outcomes, and adversarial selection. Deterministic minimax is excluded because it neither models chance nodes nor answers the stochastic-search question.

The shared evaluation is a weighted difference between the players: score (100), objective-cell control (10), distance to the nearest objective (2), capture threats (15), net-launcher potential (20), swap potential (8), piece safety (10), and legal-action mobility (1). The dominant score weight encodes the game objective; all weights are fixed across final conditions and stored in every mode configuration.

Experimental design and metrics

Each board/stopping-rule condition contains every ordered non-self pair among six agents and three matched game seeds (90 games). Mirrored orders reuse the seed, allowing direct first-versus-second comparison. Four independent deck conditions contribute 360 games, but the default fixed-horizon control is shared with the board study; the 12 unique main/deck modes therefore contain 1,080 games. A bounded goal-20 depth study adds 36 games across the three boards, for 1,116 total.

A win contributes one win point, a draw one half, and a loss zero. The engine measures each action-selection call with a monotonic high-resolution clock; runtime is total logged decision time divided by non-pass decisions. The corrected suite ran sequentially on an Intel Core i5-9600K under 64-bit Python 3.12.13 and Windows 10 (build 19045), so timing supports within-machine comparisons rather than portable benchmarks. Behavioral analysis reconstructs adjacent and net-launcher captures from per-turn logs. Percentages are descriptive: a direct main matchup has six games, and the same seeds recur across conditions. Each mode stores its

complete configuration. All 1,116 logs replayed from forced draws matched the recorded final outcomes.

Analysis plan

The primary comparison asks which agents perform well across boards and stopping rules. Secondary analyses examine first-versus-second order, performance per unit decision time, isolated deck composition, direct expectimax-depth matchups, and adjacent versus net-launcher captures. The deck study is analyzed separately because it was deliberately not crossed with board geometry.

No null-hypothesis significance tests or confidence intervals are reported. Games within a matched seed and repeated conditions are not independent, while direct matchups contain only six games. The analysis therefore reports denominators, matched contrasts, and calibrated descriptive conclusions; confirmatory inference is left to a larger independent-seed experiment.

Experimental Results

Overall performance

Across the 12 unique main and deck conditions, the aggregate order was Rule (66.0%), Expectimax d3 (63.6%), MCTS (62.4%), Expectimax d2 (56.8%), Greedy (51.0%), Random (0.3%). Rule therefore provided the strongest observed performance-cost trade-off at its measured budget. This pooled ranking is descriptive rather than universal: board geometry and stopping rule changed the relative ordering (Figure 3).

Board geometry and stopping rule

The heatmaps show that no pooled percentage captures every condition. Goal-10 games were especially short on the four-point inner-ring board, where one or two successful center occupations can determine the result. Mean combined duration

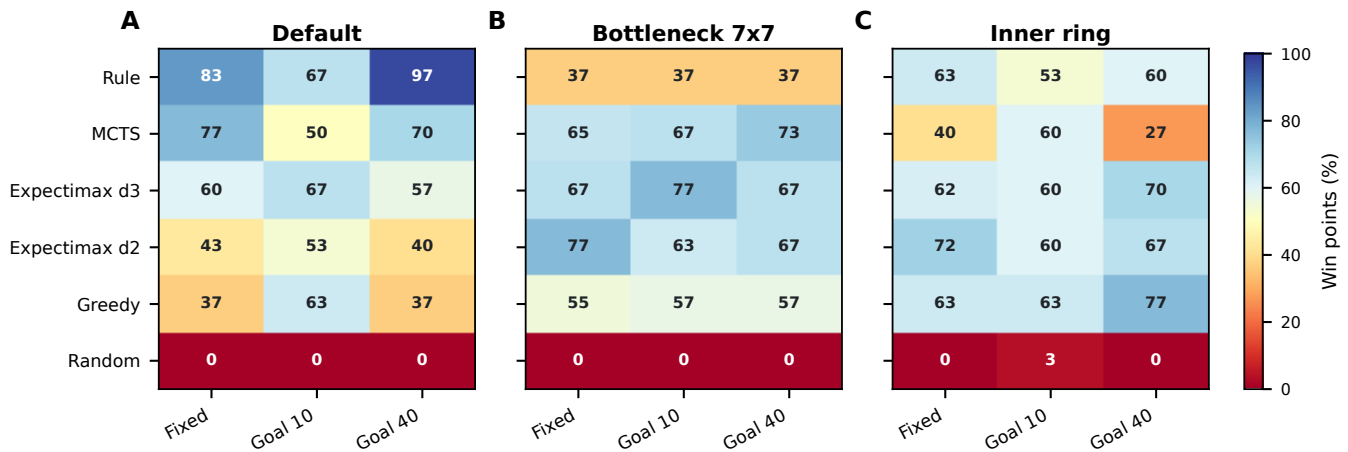


Figure 3: **Agent rankings vary across board geometry and stopping rule.** (A-C) Win points for the six agents on the default, bottleneck, and inner-ring boards, respectively. Columns are a fixed 25-turn-per-player horizon and point goals of 10 or 40; the turn cap remains a safety bound in goal games. Each cell contains 30 agent-game observations: five opponents, both player orders, and three matched seeds. A win contributes 1, a draw 0.5, and a loss 0. Color and printed values both encode win points (%); 50% denotes parity across opponents. Conditions are separate tests, not pooled replicates.

Table 1: Aggregate results across the 12 unique main/deck conditions.

Agent	N	Win points	Margin	ms/decision
Rule	360	66.0%	+10.31	4.76
Expectimax d3	360	63.6%	+8.37	956.34
MCTS	360	62.4%	+6.36	166.94
Expectimax d2	360	56.8%	+6.77	110.88
Greedy	360	51.0%	+5.30	9.42
Random	360	0.3%	-37.11	<0.01

was 13.8, 15.5, and 6.7 turns for goal 10 on the default, bottleneck, and inner-ring boards, respectively, compared with 58.5, 35.0, and 22.4 turns for goal 40. Goal 10 is therefore an opening-focused stress test rather than a substitute for longer evaluation.

Runtime and move order

Performance did not increase monotonically with decision time (Figure 4A). Measured costs were 4.8 ms for rule, 9.4 ms for greedy, 110.9 ms for expectimax depth 2, 166.9 ms for MCTS, and 956.3 ms for expectimax depth 3. Depth 3 was 8.6 times slower than depth 2, while their aggregate win-points difference was +6.8 percentage points. Additional lookahead changed decisions, but its cost was not matched by a uniform direct advantage.

Move-order effects were smaller than the full spread between agents but remained visible (Figure 4B). The largest absolute first-minus-second contrast was +8.1 percentage points for MCTS; MCTS was +8.1, depth-3 expectimax -0.6, and rule +4.7. Because every pair is mirrored with the same seed, these contrasts control opponent identity and the seeded refill process more closely than an unmatched tournament.

Independent deck-composition test

Changing card frequencies shifted relative performance even though board, stopping rule, market refresh, and seed design were fixed (Figure 5). Rule led every deck (81.7-93.3%), but the mid-ranking reordered: depth-2 expectimax ranged from 40.0% under the movement-heavy deck to 53.3% under the capture-heavy deck and fell behind greedy when movement cards dominated. Thus a ranking from one action distribution should not be generalized to the game family. Because deck and board were not crossed, this experiment estimates deck effects on the default board, not deck-by-geometry interactions.

Expectimax depth

Across nine direct board/stopping-rule cells, depth 3 obtained 52.8% of win points against depth 2 over 54 agent-game observations (Figure 6A). The bounded study shows the computational reason for stopping at depth 4: runtime rises steeply on all boards (Figure 6B), while the corresponding performance estimates use only one seed and are unstable (Figure 6C). Depth 5 exceeded 120 seconds for one benchmark decision; depth 6 was therefore not run. The branching structure and approximate top-2 beam make this result plausible, but the experiment does not isolate a causal mechanism and does not establish search pathology.

Dominant strategies and net-launcher use

Rule is explicitly capture-first and objective-first, whereas greedy and expectimax optimize a broader multi-term heuristic that can trade immediate score for mobility, safety, or formation potential. In the main suite, Move 1 was every agent's most-used card (24.8-32.5% of played actions), consistent with its 8/28 deck prevalence. Capture-card shares, by contrast, separated the agents: random played captures in only 3.1% of its actions against 16.4-22.7% for the

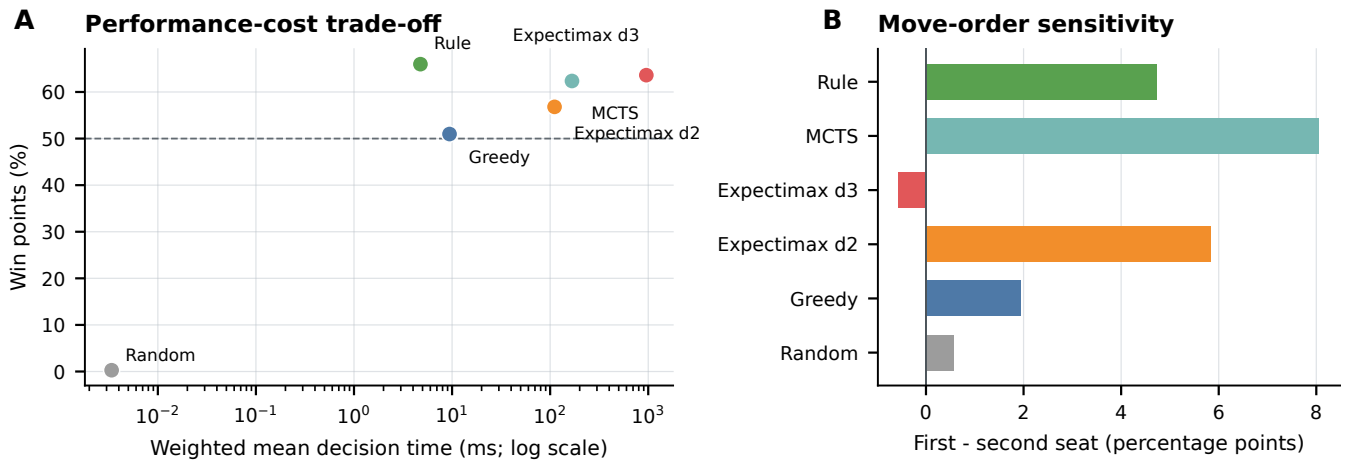


Figure 4: **Additional computation does not yield a monotonic performance gain.** (A) Aggregate win points versus weighted mean decision time across the 12 main and independent-deck conditions. Each agent has 360 agent-game observations; time is total recorded decision time divided by non-pass decisions and is shown on a logarithmic axis. The dashed line marks 50% win points. (B) First-seat minus second-seat win-points rate for the same conditions (180 agent-game observations per seat and agent). Positive values favor moving first. Colors identify agents in both panels; labels provide a redundant non-color encoding.

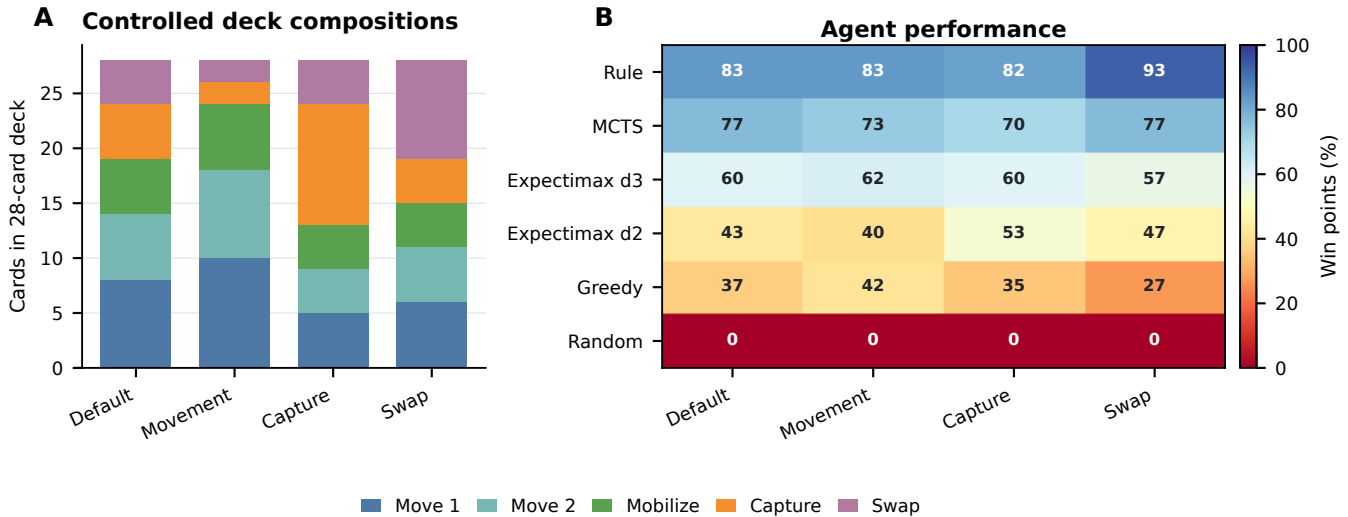


Figure 5: **Independently changing deck composition shifts relative agent performance.** (A) Counts of the five action cards in each 28-card deck; stacked segments sum to 28. (B) Win points on the default board under the fixed 25-turn-per-player horizon. Each cell contains 30 agent-game observations across five opponents, both player orders, and three matched seeds. Color and printed values encode win points (%). The default condition is the shared control; the three alternatives change only card counts while retaining the two-pass market refresh.

other agents, and capture rates ranged from 0.53 (random) to 4.26 (depth-3 expectimax) per agent-game. Deliberate capture seeking therefore distinguishes every non-random agent from the random baseline, but these summaries still do not isolate a winning mechanism; that requires policy and heuristic-weight ablation. Complete action counts are available in the accompanying results archive (tables/16_action_usage_summary.csv).

The net launcher accounted for 1,424 of 7,302 captures (19.5%) across the complete suite. In the comparable 1,080-

game main/deck suite, 1,367 of 7,155 captures were net launches (19.1%; Figure 7). Net use also varied by agent: rule fired nets in 9.5% of its captures, against 17.7-23.5% for MCTS, greedy, and the expectimax variants. This frequency establishes that the mechanic affects play, but event counts alone do not establish that using it causes wins.

Experimental Conclusions and Summary

The project achieved its objective of constructing a reproducible stochastic game domain and comparing bounded

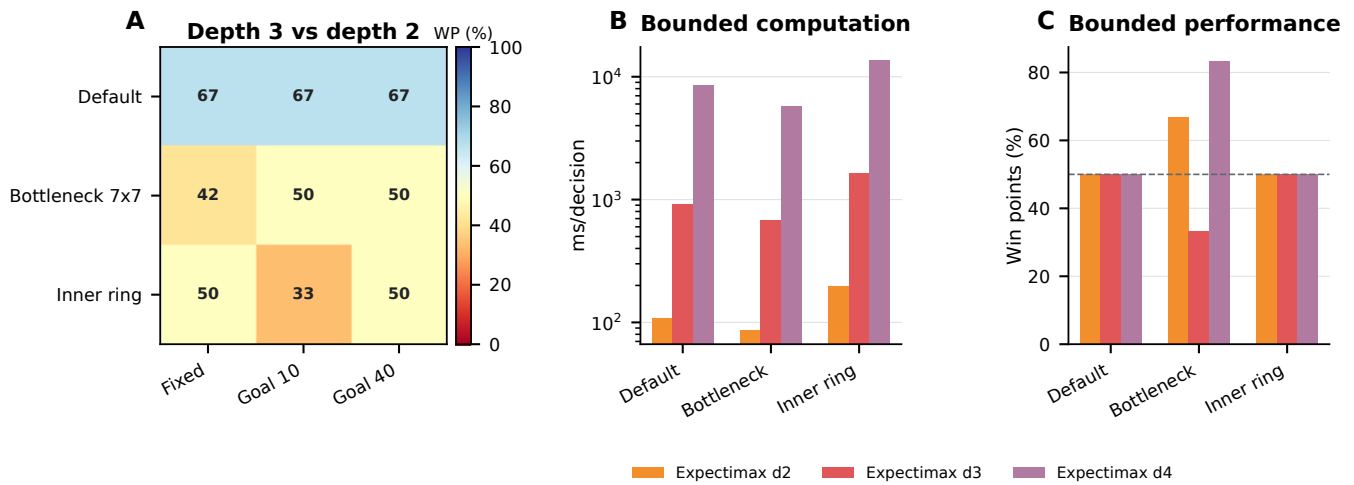


Figure 6: Deeper expectimax is substantially slower and is not uniformly stronger. (A) Depth-3 win points in direct depth-3 versus depth-2 matches for all nine board/stopping-rule combinations. Each cell summarizes six games (three matched seeds in both player orders); 50% denotes parity. (B-C) Weighted mean decision time and win points, respectively, in the bounded goal-20 feasibility study. Bars summarize six agent-game observations per board and depth; all variants use a top-2 action cap. Runtime uses a logarithmic axis and the dashed performance line marks 50%. Depth 4 used one seed and is interpreted as a feasibility measurement; depth 5 exceeded 120 s for one benchmark decision and depth 6 was not run.

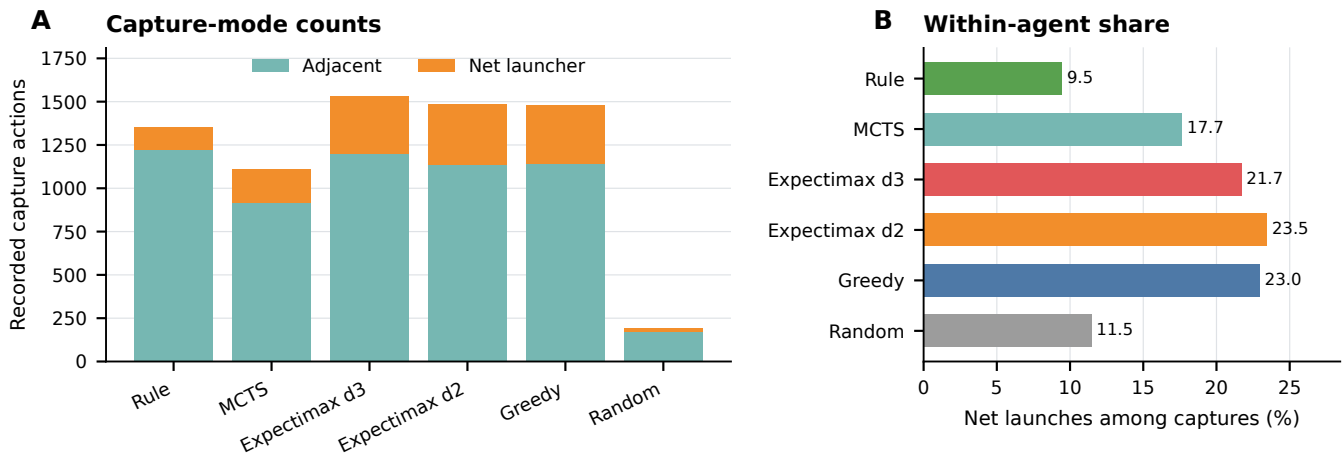


Figure 7: Net launches are a recurring but minority capture mode. (A) Adjacent and net-launcher capture actions reconstructed from all per-turn logs in the 1,080-game main and deck suite (7,155 captures; 1,367 net launches). Counts are events, not games, and therefore reflect both opportunity and agent behavior. (B) Net launches as a percentage of each agent’s captures, with exact percentages printed beside the bars. Expectimax depth 4 is excluded because it appears only in the smaller bounded feasibility study. Across the displayed agents, the net share ranges from 9.5% (Rule) to 23.5% (Expectimax d2).

search implementations with meaningful baselines. Under the tested settings, Rule produced the highest aggregate win-points rate, Expectimax d3 led the evaluated search agents, and depth-3 expectimax was substantially more expensive than depth 2 without a uniform direct advantage. Board geometry, score goal, deck composition, and player order all changed outcomes, so one pooled tournament ranking would obscure important structure.

The main limitations are statistical and budgetary. Direct depth matchups contain six games per condition, only three

seeds support the main cells, seeds recur across conditions, and no confidence intervals are claimed. Decks were varied independently rather than factorially with boards. Expectimax retains a heuristic top-2 beam, depth 4 uses one seed, and MCTS samples rather than exhaustively expands chance outcomes. MCTS and expectimax were not matched by wall-clock budget, and the shared heuristic was not tuned on a separate training set. The corrected agents pass hidden-deck-order invariance tests, but the API still carries the execution deck, so architectural separation would be safer than relying

on tested discipline. Future work should increase independent paired seeds, compare equal-time performance curves, ablate heuristic terms, and test *-minimax or progressive-widening variants where chance branching grows.

Code and data availability

Source code, board and deck definitions, experiment configurations, per-game results, move logs, analysis scripts, and figure-generation code are available at github.com/Naharda/Grid_Control_Game. The report figures are generated by `analysis/build_course_paper.py`; complete final-suite tables are indexed under `final_results/`.

References

- Ballard, B. W. 1983. The *-minimax search procedure for trees containing chance nodes. *Artificial Intelligence*, 21(3): 327–350.
- Browne, C. B.; Powley, E.; Whitehouse, D.; Lucas, S. M.; Cowling, P. I.; Rohlfshagen, P.; Tavener, S.; Perez, D.; Samothrakis, S.; and Colton, S. 2012. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1): 1–43.
- Coulom, R. 2006. Efficient selectivity and backup operators in Monte-Carlo tree search. In *Computers and Games*, 72–83.
- Genesereth, M.; Love, N.; and Pell, B. 2005. General game playing: Overview of the AAAI competition. *AI Magazine*, 26(2): 62–72.
- Issakkimuthu, M.; Fern, A.; and Tadepalli, P. 2020. The choice function framework for online policy improvement. In *Proceedings of AAAI*, 10178–10185.
- Kearns, M.; Mansour, Y.; and Ng, A. Y. 1999. A sparse sampling algorithm for near-optimal planning in large Markov decision processes. In *Proceedings of IJCAI*, 1324–1331.
- Kocsis, L.; and Szepesvari, C. 2006. Bandit based Monte-Carlo planning. In *ECML*, 282–293.
- Lanctot, M.; Saffidine, A.; Veness, J.; Archibald, C.; and Winands, M. H. M. 2013. Monte Carlo *-minimax search. In *Proceedings of IJCAI*, 580–586.
- Pearl, J. 1983. On the nature of pathology in game searching. *Artificial Intelligence*, 20(4): 427–453.